GenAI Report: EECS 348 Assignment 4

Kaleb Justice Blakemore
EECS 348
October 13, 2026

Name of Two GenAI Programs

- ChatGPT
- Copilot

How They Were Accessed

ChatGPT: Accessed through a web browser at chatgpt.com by entering a prompt into the chat interface.

Copilot: Accessed through the chat panel in the web browser while working on the program.

Prompt Used

The same prompt was given to both programs so I could compare their results using the same instructions.

Write a complete, single-file C++ object-oriented program to solve five Sudoku puzzle text files (puzzle1.txt through puzzle5.txt) using a recursive depth-first search with backtracking. The program must define a class with member functions for loading puzzles, validating constraints (rows, columns, and 3x3 blocks), printing boards, and recursively solving the grid. Use objects rather than standalone functions, and include a main function that processes all five files sequentially, printing the initial state and either the solved grid or 'No solution found'.

Generated Code Overview

ChatGPT: ChatGPT generated a program using a SudokuSolver class with separate functions for loading the puzzle, checking the numbers, printing the board, and solving the puzzle. The code followed the requested object-oriented structure and used recursion to work through the empty cells.

```cpp
/*
 * Program Name: EECS 348 Assignment 4 – Sudoku Solver
```

```
 * Description: Solves Sudoku puzzles using recursion and backtracking.
 * Input: puzzle1.txt through puzzle5.txt
 * Author: Kaleb Justice Blakemore
 * Date: October 15, 2026
 */

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class SudokuBoard {
private:
    int grid[9][9];

public:
    // Initialize the board with empty cells.
    SudokuBoard() {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                grid[row][col] = 0;
            }
        }
    }

    // Load a puzzle from a file. Underscores represent empty cells.
    bool loadPuzzle(const string& filename) {
        ifstream file(filename);

        if (!file.is_open()) {
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                char ch;

                if (!(file >> ch)) {
                    file.close();
                    return false;
                }

                if (ch == '_') {
                    grid[row][col] = 0;
                } else if (ch >= '1' && ch <= '9') {
                    grid[row][col] = ch - '0';
                } else {
```

```
                file.close();
                return false;
            }
        }
    }

    file.close();
    return true;
}

// Print the current Sudoku board.
void printBoard() const {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == 0) {
                cout << "_ ";
            } else {
                cout << grid[row][col] << " ";
            }
        }
        cout << endl;
    }
}

// Check whether a number can be placed in a cell.
bool isValid(int row, int col, int num) const {
    // Check the row and column.
    for (int i = 0; i < 9; i++) {
        if (grid[row][i] == num ||
            grid[i][col] == num) {
            return false;
        }
    }

    // Find the starting position of the 3x3 block.
    int startRow = row - row % 3;
    int startCol = col - col % 3;

    // Check the 3x3 block.
    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (grid[r][c] == num) {
                return false;
            }
        }
    }

    return true;
```

```
    }

    // Check whether the original puzzle follows Sudoku rules.
    bool isBoardValid() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                int num = grid[row][col];

                if (num != 0) {
                    // Temporarily remove the number before checking.
                    SudokuBoard temp = *this;
                    temp.grid[row][col] = 0;

                    if (!temp.isValid(row, col, num)) {
                        return false;
                    }
                }
            }
        }

        return true;
    }

    // Solve the puzzle using recursive backtracking.
    bool solveRecursive(int row, int col) {
        // All rows have been completed.
        if (row == 9) {
            return true;
        }

        // Move to the next row when the current row is finished.
        if (col == 9) {
            return solveRecursive(row + 1, 0);
        }

        // Skip cells that already contain numbers.
        if (grid[row][col] != 0) {
            return solveRecursive(row, col + 1);
        }

        // Try numbers 1 through 9.
        for (int num = 1; num <= 9; num++) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;

                // Continue solving the next cell.
                if (solveRecursive(row, col + 1)) {
                    return true;
```

```
                }

                // Undo the guess if it leads to a dead end.
                grid[row][col] = 0;
            }
        }

        // No number worked, so go back to the previous cell.
        return false;
    }

    // Start solving from the top-left corner.
    bool solve() {
        if (!isBoardValid()) {
            return false;
        }

        return solveRecursive(0, 0);
    }
};

int main() {
    string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process all five puzzle files in order.
    for (int i = 0; i < 5; i++) {
        cout << "Puzzle File: " << puzzleFiles[i] << endl;
        cout << "-----------------------------------" << endl;

        SudokuBoard board;

        if (!board.loadPuzzle(puzzleFiles[i])) {
            cout << "Could not load puzzle file or invalid file format."
                 << endl << endl;
            continue;
        }

        cout << "Initial Puzzle:" << endl;
        board.printBoard();
        cout << endl;

        cout << "Solution:" << endl;
```

```
        if (board.solve()) {
            board.printBoard();
        } else {
            cout << "No solution found" << endl;
        }

        cout << "\n==================================\n" << endl;
    }

    return 0;
}
```

Copilot: Copilot generated a similar program, but some of its helper functions were organized differently. I found that its code needed some changes to

better match the assignment's requirement to organize the program around a class.

```
#include <iomanip>

using namespace std;

class SudokuSolver {
private:
    vector<vector<int>> board;

public:
    SudokuSolver() {
        board.resize(9, vector<int>(9, 0));
    }

    bool loadPuzzle(const string& filename) {
        ifstream infile(filename);

        if (!infile) {
            cerr << "Error: Cannot open file " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (!(infile >> board[row][col])) {
                    cerr << "Error: Invalid puzzle format in "
```

```
                    << filename << endl;
                return false;
            }
        }
    }

    infile.close();
    return true;
}

void printBoard() const {
    for (int row = 0; row < 9; row++) {
        if (row % 3 == 0 && row != 0) {
            cout << "------+-------+------" << endl;
        }

        for (int col = 0; col < 9; col++) {
            if (col % 3 == 0 && col != 0) {
                cout << "| ";
            }

            cout << board[row][col] << " ";
        }
        cout << endl;
    }
}

bool isValid(int row, int col, int num) const {
    // Check row
    for (int c = 0; c < 9; c++) {
        if (board[row][c] == num) {
            return false;
        }
    }

    // Check column
    for (int r = 0; r < 9; r++) {
        if (board[r][col] == num) {
            return false;
        }
    }

    // Check 3x3 block
    int startRow = row - row % 3;
    int startCol = col - col % 3;

    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
```

```
                if (board[r][c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    return true;
                }
            }
        }
        return false;
    }

    bool solve() {
        int row, col;

        if (!findEmptyCell(row, col)) {
            return true; // Puzzle solved
        }

        for (int num = 1; num <= 9; num++) {
            if (isValid(row, col, num)) {
                board[row][col] = num;

                if (solve()) {
                    return true;
                }

                // Backtrack
                board[row][col] = 0;
            }
        }

        return false;
    }
};

int main() {
    const string puzzleFiles[] = {
        "puzzle1.txt",
        "puzzle2.txt",
```

```
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const string& filename : puzzleFiles) {
        cout << "\n========================================\n";
        cout << "Processing: " << filename << endl;
        cout << "========================================\n";

        SudokuSolver sudoku;

        if (!sudoku.loadPuzzle(filename)) {
            cout << "Skipping file.\n";
            continue;
        }

        cout << "\nInitial Puzzle:\n";
        sudoku.printBoard();

        if (sudoku.solve()) {
            cout << "\nSolved Puzzle:\n";
            sudoku.printBoard();
        } else {
            cout << "\nNo solution found.\n";
        }

        cout << endl;
    }

    return 0;
}
```

Code Analysis

Correctness: Both programs followed the general idea of solving Sudoku through backtracking. The program tries numbers in empty cells, checks whether they follow the Sudoku rules, and moves on if they are valid. If a number does not work later in the puzzle, the program goes back and tries another number. ChatGPT's version was easier for me to follow, while Copilot's version needed some adjustments to its 3x3 block checks. The final program also checks whether each puzzle file can be opened before trying to solve it.

Execution Time: Both programs use the same basic method to solve the puzzles. The time needed depends on how difficult each puzzle is and how many numbers the program has to try. Since the program only works with a 9x9 Sudoku board, the amount of work is manageable for typical puzzles. I would need to measure both programs under the same conditions to make a fair comparison of their actual running times.

Space Complexity: Both programs store the Sudoku board in a 9x9 grid and use recursion to keep track of the steps taken while solving it. The program changes empty cells as it works through the puzzle and resets them when a number does not work. This means it does not need to create a new copy of the entire board every time it tries a number.

Maintainability: I found ChatGPT's code easier to follow because the main parts of the program were separated into different class functions. For example, loading the puzzle, checking the numbers, printing the board, and solving the puzzle each have their own functions. This made it easier to understand what each part was responsible for and make changes when needed. Copilot's code had a similar overall goal, but I preferred how ChatGPT organized the program.

Selection and Justification

Selected Program: ChatGPT

I selected ChatGPT because I found its code easier to understand and work with. Its class structure matched the assignment requirements, and the separate functions made it easier to follow how the program worked. I also found it easier to make changes without having to reorganize as much of the code. Overall, ChatGPT gave me a better starting point for completing the assignment.

Code Improvements Made

File Handling: Added a check to make sure each puzzle file could be opened before the program tried to read it. If a file cannot be opened, the program displays an error message and moves on to the next file.

Reading the Puzzle: Added handling for underscores so that empty cells are stored as zero in the program and printed as underscores on the board.

Checking the 3x3 Blocks: Used a calculation to find the starting row and column of each 3x3 block. This makes it easier to check the correct section of the board without adding unnecessary conditions.

Solving the Puzzle: Kept the solving process within the SudokuBoard class. The program tries numbers from 1 through 9, moves forward when a number works, and resets the cell when it needs to try another number. This keeps the solving process organized and avoids making unnecessary copies of the board.

Comments and Documentation: Added information at the beginning of the program and comments throughout the code to explain what the main sections and functions do. This makes the program easier to read and understand when reviewing it later.